PREDICTING ISOFORM EXPRESSION FROM GENE-LEVEL PROFILES USING REPRESENTATION LEARNING

Konstantina Gkopi (s243692), Henrik S. J. Knudsen (s215065), Lucía de Lamadrid Ordóñez (s243311), Eric Torres García (s243275) and Jorge Santiago Bajo (s243310).

ABSTRACT

Despite the prevalence of the one-gene-one-protein model, the process of alternative splicing can yield multiple isoforms from the same gene, often with different functions and biological relevance in different processes. Therefore, it is of great interest to develop computational methods to measure isoform expression. In this work, different architectures are explored to predict isoform expression from gene expression data, and the effect of different feature representation methods in the model performance is analysed. The results showed that, even if dimensionality reduction reveals useful in reducing the computational workload, further hyperparameter tuning and verification in new datasets is still necessary to achieve better performance.

1. INTRODUCTION

Isoform-level expression quantification remains challenging due to the limitations of short-read RNA-Seq, often not able to produce accurate abundance estimates [1]. Long-read sequencing technologies solve this problem, but their higher cost and the abundance of short-read RNA-Seq datasets demand the development of computational approaches to predict isoform-level expression. Despite the growing number of papers using deep learning to solve biological tasks, no publication addressing this issue was found; however, deep learning methods have already been explored for related problems, such as splicing prediction or denoisification of single-cell data using Variational Autoencoders (VAEs) [2, 3]. In addition to that, available representation learning methods and pre-trained transformer-based models, such as scGPT or BulkFormer can help to learn complex nonlinear relationships between genes, which can be useful in our mission.

The aim of this project is to build a computational method to predict isoform expression from (i) raw gene expression data and (ii) different feature representation techniques. Additionally, we tested several architectures, such as residual networks and shallow transformers to compare them against the baseline feed-forward neural network (FFNN).

2. METHODS & DATA

The gene and transcript expression datasets were sourced from the ARCHS4 database, that hosts RNA-Seq data from human and mouse. We were provided datasets from bulk and scRNA-seq. However, computational and time constraints limited the analysis to the bulk RNA-Seq data. The dataset comprised 19,882 samples covering 45,263 genes and 162,009 transcripts. Data was split into train (70%), validation (15%) and test sets (15%). A seed was used to keep reproducibility and consistency across the different models. All code used can be found in our GitHub repository.

The data was processed using a pseudocount of 1: $\log(1p) = \log(\text{count} + 1)$. Figure 5 (Appendix A) shows the distribution of the log1p count values in the bulk datasets.

2.1. Feed-forward Neural Network (FFNN)

A Feed-Forward Neural Network was used as baseline model to learn from the full high-dimensional gene expression dataset. FFNNs were also used to evaluate impact of the different feature representation methods on predictive power. Random search was used for hyperparameter optimization, which included learning rate, number of layers, hidden layer sizes, dropout rates, batch size and activation functions. Randomly sampling across the defined options allowed us to identify good-performing configurations without the computational expense of running a full grid-search. Early stopping further reduced running time as training stops if performance drops. This approach has been shown to be effective for high-dimensional search spaces, where only a subset of hyperparameters strongly influences performance [4]. Further description of the hyperparameter options is included in the deliverables of the project. The chosen optimizer was AdamW, a modified Adam (Adaptive Moment estimation) with decoupled weight decay; using Mean Squared Error (MSE) as loss function.

The execution of the FFNN runs in two stages. The first stage trains 100 neural networks with different combinations of hyperparameters for 50 epochs. The five combinations with the lowest validation error are selected for the second stage, where they are trained for 200 epochs. The parameter

selection as well as the number of epochs were chosen based on the course notebooks[1].

2.2. Feature representation methods

RNA-Seq data can be noisy due to batch effects, like sample collection and library preparation, but also by the fact that more than 40% of entries in the datasets are 0.

Therefore, a number of different methods were used for feature representation. The resulting reduced-dimensionality features were then fed into the baseline FFNN with the same procedure as previously described to predict transcript expression.

2.2.1. Principal Component Analysis (PCA)

PCA is one of the main techniques for dimensionality reduction. It transforms the data into new features called principal components. PCA was applied to the input raw gene expression data after it was scaled.

2.2.2. Variational Autoencoders (VAEs)

A variational autoencoder (VAE) is an unsupervised learning model designed to learn a compact latent representation of the data. In this work, a VAE was used to generate low-dimensional embeddings, based on code adapted from the Jupyter Notebook 7.2[1] and modified to better suit the characteristics of our dataset. The architecture consists of an encoder, which maps the input into a latent feature space, and a decoder, which reconstructs the input from this representation. Both components are implemented as fully connected three-layer neural networks with ReLU activation functions. The prior and approximate posterior are defined as a Gaussian distribution, whereas the observation model is a negative binomial, typically used in expression data [5]. The loss function is computed through a modified ELBO.

2.2.3. BulkFormer

BulkFormer[2] is a large-scale foundation model for bulk transcriptome analysis, containing more that 150 million parameters covering around 20,000 protein-encoding genes [6]. The choice of BulkFormer as the pre-trained transformer model used in this project is due to it being one of the few foundation models designed on bulk RNA-Seq data. However, the lack of consideration of the non-coding genes can be a potential caveat, as many of these genes such as miRNAs act as regulatory elements [7]. This model incorporates a hybrid encoder architecture, combining a graph neural network to capture gene-gene interactions and a performer module to model global expression dependencies [6]. The result of this model is an embedding of size 640 per sample, which is a significant dimensionality reduction compared to the original 45,263 genes.

2.2.4. Shallow Transformer

Self-attention to the input of 45,000 genes would cause a computational cost of O(N^2), making a standard transformer infeasible; therefore, the model groups genes into fixed-size patches, following the strategy used in Vision Transformers [8]. Each patch was projected into a d_{model}-dimensional token embedding and augmented with a learnable positional embedding. Since genes have no canonical biological ordering, positional embeddings do not encode sequence structure; instead, they break symmetry between tokens and stabilize training, preventing the transformer from treating all positions as identical. The sequence of patch embeddings is processed by 1–2 Transformer Encoder layers (multi-head self-attention, feed-forward network, residual connections, and LayerNorm) [9], inspired by the Jupyter Notebook 6.1[1]. The patch sequence is reduced into a single sample-level representation and the resulting representations were pooled into a single sample-level embedding. This vector was fed into a fixed MLP head:

$$\begin{aligned}\text{LayerNorm} \to \text{Linear}(d_{\text{model}} \to h) \to \text{GELU/ReLU} \\ \to \text{Dropout} \to \text{Linear}(h \to 162\text{k})\end{aligned}$$

which provides sufficient nonlinearity to decode the pooled embedding without dominating the parameter count.

3. RESULTS AND DISCUSSION

The best performing FFNN configuration, hyperparameters and performance metrics are presented in Table 1.

3.1. Feed-Forward Network with raw data

The best performing configuration for the raw gene expression data was a shallow network with 1024 hidden units (Table 1). It is noteworthy that all top five configurations were also shallow neural networks, with most models using 1024 units (Figure 1). This suggests that relatively simple architectures provide a strong baseline for predicting isoform expression from raw gene expression data.

The FFNN architecture was extended by incorporating residual blocks, each consisting of two linear transformations with optional batch normalization and non-linear activation. When evaluating the model on the raw gene-expression, performance decreased slightly compared to the non-residual baseline ($MSE_{test} = 0.94$, $\rho_{test} = 0.717$). This behavior is not entirely unexpected: residual connections are mostly beneficial in very deep networks, where they help prevent vanishing gradients and stabilize optimization. In shallower architectures these issues are minimal, so skipping connections offers no improvement [10].

[1] https://github.com/DeepLearningDTU/02456-2025.git
[2] https://github.com/KangBoming/BulkFormer

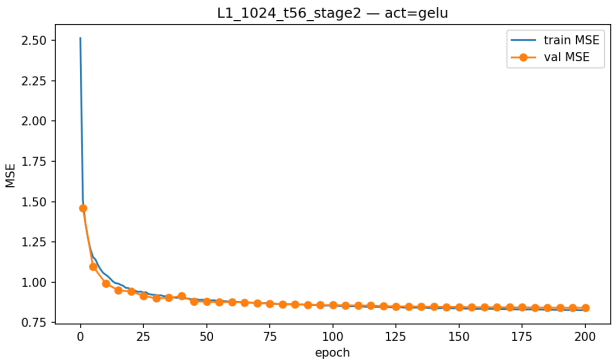


Fig. 1: Training and validation curves for the best performing baseline FFNN using raw data as input.

3.2. PCA

To choose the number of dimensions, a series of PCA-based FFNN models were trained using varying numbers of PCA input components while keeping all other hyperparameters fixed. A dimensionality of 1000 principal components yielded the best performance (Figure 6 in Appendix A).

The hyperparameter configuration that produced the best PCA-based FFNN model is presented in Table 1, and the corresponding training curve is shown in Figure 2. A dimensionality reduction from 45,263 to 1,000, in combination with hyperparameter optimization led to a $MSE_{test} = 0.859$, quite similar to the baseline FFNN model ($MSE_{test} = 0.849$) despite the dimensionality reduction. The PCA-based modeling approach achieves substantial data compression with minimal degradation in predictive performance.

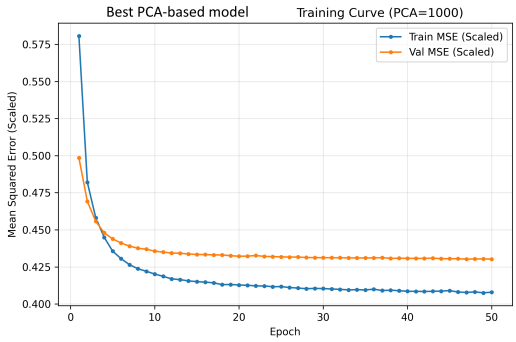


Fig. 2: Training curve for the best PCA-based FFNN model. Y-axis represents scaled MSE.

3.3. VAEs

Attempts to improve the performance obtained with raw gene expression using low-dimensional VAE embeddings revealed some challenges. Given the impossibility to perform full-grid search of hyperparameters, a more guided and systematic strategy was adopted, in which individual factors were varied one at a time. None of them achieved to outperform the baseline model.

Due to gradient explosion and training instability when using the learning rate from course notebook, it was reduced (from 10^{-3} to $3 \cdot 10^{-5}$). This stabilized training but the performance in the FFNN deteriorated badly compared to the baseline FFNN. Increasing the number of epochs did not resolve this, as training and validation curves diverged, indicating overfitting. To address this, and following the methods found in the literature [11], we implemented a dropout rate of 0.1 after the input layer, which helped improving performance although it remained below the baseline.

Additionally, KL collapse was observed, a problem we tackled using β-annealing (from 0.0 to 1.0) with 50 warm-up epochs, as described by other researchers [12].

For choosing the latent features' size, a network structure (640, 160 and 80) based on [13] was first tested. Following TA's recommendations, we trained VAEs with different latent features' sizes keeping the rest of hyperparameters fixed, and used the embeddings generated to feed a simple FFNN with 12 random hyperparameter configurations. At this step, we also increased the depth of the encoder and decoder with one more layer due to the high dimensionality of the data. The latent representation with the best performance was selected for subsequent analyses (Figure 3).

Despite trying different combinations of hyperparameters, the model performed consistently worse than the baseline model. This also happened even using a semi-supervised learning approach, in which VAE loss was augmented with an auxiliary isoform-prediction term. This might be due to the sharp reduction in dimensionality (a decrease of more than 99% of the initial dimensions). However, despite lower performance, achieving comparable predictions after compressing over 45,000 genes into fewer than 100 latent features is already a remarkable result.

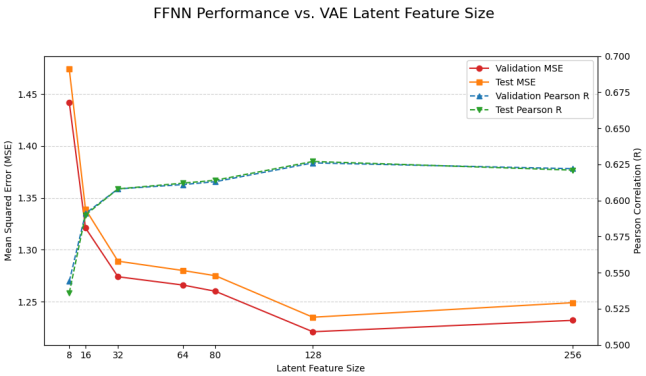


Fig. 3: Performance of the FFNN based on the latent feature size.

Table 1: Summary of the best configurations for different input data. **Abbreviations:** Act. (activation), BN (batch norm. size), LR (learning rate), MSE (mean squared error), ρ (Pearson correlation), *[p (patch size), d (embedding dimensions)].

Data input	Arch.	Best config.	Act.	Dropout	BN	LR	Val. MSE	Val. ρ	Test MSE	Test ρ
Raw data	FFNN	L1_1024	GELU	0.2	128	0.001	0.842	0.746	0.849	0.748
Raw data	ResNet	L3_256_2048_2048	GELU	No	192	0.001	0.929	0.715	0.94	0.717
Raw data	ShallTrans	L2_p32_d256*	ReLU	No	64	0.003	0.919	0.719	0.929	0.721
PCA	FFNN	L1_512	Tanh	No	128	0.003	0.866	0.745	0.859	0.743
BulkFormer	FFNN	L2_256_2048	GELU	0.2	128	0.002	0.971	0.705	0.979	0.707
VAE	FFNN	L2_2048_2048	ReLU	No	No	0.001	1.198	0.633	1.208	0.635

3.4. Transformers

As observed in Table 1, the configuration using BulkFormer's embeddings with best performance was two hidden layers with 256 and 2048 units, using similar hyperparameters as the baseline FFNN. Despite not improving the performance, it did not decreased dramatically. The worse metrics could be due to loss of information, attributed to the compression into embeddings, and the filtering out of non-coding genes that play an important part in expression regulation [7].

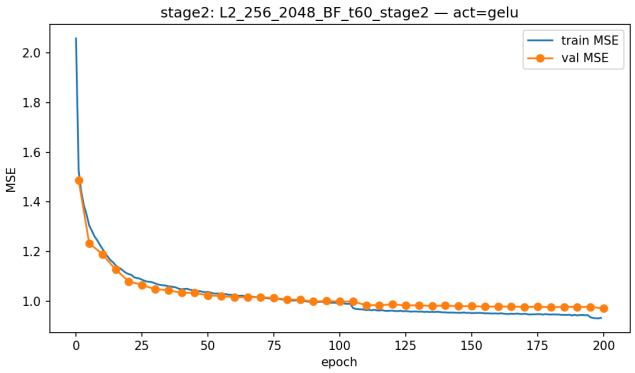


Fig. 4: Training and validation curve for the best performing FFNN using BulkFormer embeddings as input.

To enhance the performance of this model, the last two layers of BulkFormer were fine-tuned by linking the loss from the highest-performing FFNN model using BulkFormer embeddings as input to the weights of in the last two transformer layers. The choice of using the last two layers was based on the impossibility of fine-tuning every parameter due to the huge size of the model. This did not work as expected, resulting in a big increase in MSE. Modifying the two last layers of the model might not be enough or might even disrupt the model. Additionally, it must be noted that, due to the computational complexity of BulkFormer and the GPU memory size, fine-tuning was performed with a batch size of 1 rather than 128, as used in the FFNN trained on BulkFormer embeddings. This reduction in batch size could also contribute to decreasing the performance.

3.4.1. Shallow Transformers

Patch sizes (16–64 in the large search; up to 256 in the small search) were varied to control gene-grouping granularity and sequence length. Embedding dimensions ($d_{\text{model}} = 64$–384), numbers of attention heads (2–8), and depths (1–2 layers) were explored to adjust model capacity within GPU limits. Pooling strategies (mean or attention), activation functions (GELU or ReLU), and standard training hyperparameters, including learning rate (1–$3 \cdot 10^{-3}$), dropout (0–0.2), and batch size (8–64), were also included in the search.

The initial 10-trial search, constrained to smaller models, yielded a best configuration with patch size 64, $d_{\text{model}} = 128$, 4 heads, and attention pooling, achieving a test MSE of 0.973 ($\rho = 0.71$). In the expanded 50-trial search, a substantially stronger model emerged with patch size 32, $d_{\text{model}} = 256$, 8 heads, 2 layers, mean pooling, and no dropout, reaching a test MSE of 0.929 ($\rho = 0.72$).

Two clear patterns appeared across the top models: (i) smaller patches (16–32) improved performance by preserving finer gene-expression structure, and (ii) moderate-to-high embedding dimensionality ($d_{\text{model}} = 192$–256) produced more expressive representations. However, because gene-expression vectors have no inherent order, the patching scheme imposes an artificial structure that limits the transformer's advantage; which along the shallow architecture required by memory constraints, made the model not exceed the performance of the baseline FFNN, which more directly captures global gene interactions.

4. CONCLUSION

This project has evaluated different representation learning methods to predict isoform expression using bulk RNA-Seq gene expression data. Despite not finding improvements in performance in comparison to the baseline FFNN, some models achieve a similar performance with highly reduced dimensionality, such as PCA or BulkFormer. This could save computational resources without compromising performance. Further research includes testing the models on single-cell data, that due to time constraints was not possible for this project.

5. REFERENCES

[1] Dena Leshkowitz, Merav Kedmi, Yael Fried, David Pilzer, Hadas Keren-Shaul, Elena Ainbinder, and Bareket Dassa, "Exploring differential exon usage via short- and long-read rna sequencing strategies," *Open Biology*, vol. 12, no. 9, pp. 220206, 2022.

[2] Mohammad Lotfollahi, Falko A. Wolf, and Fabian J. Theis, "scgen predicts single-cell perturbation responses," *Nature Communications*, vol. 10, no. 1, pp. 5240, 2019.

[3] Hui Xiong, Babak Alipanahi, Leonard J. Lee, Hans Bretschneider, Daniele Merico, Ryan K. C. Yuen, Yu Hua, Serge Gueroussov, Hamed S. Najafabadi, Timothy R. Hughes, Quaid Morris, Yoseph Barash, and Brendan J. Frey, "The human splicing code reveals new insights into the genetic determinants of disease," *Science*, vol. 347, no. 6218, pp. 1254806, 2015.

[4] James Bergstra and Yoshua Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[5] Christopher Heje Grønbech, Maximillian Fornitz Vording, Pascal N Timshel, Casper Kaae Sønderby, Tune H. Pers, and Ole Winther, "scvae: variational auto-encoders for single-cell gene expression data," *Bioinformatics*, vol. 36, no. 16, pp. 4415—-4422, August 2020.

[6] Boming Kang, Rui Fan, Meizheng Yi, Chunmei Cui, and Qinghua Cui, "A large-scale foundation model for bulk transcriptomes," *bioRxiv*, 2025.

[7] Alex Mas Monteys, Ryan M. Spengler, Ji Wan, Luis Tecedor, Kimberly A. Lennox, Yi Xing, and Beverly L. Davidson, "Structure and activity of putative intronic mirna promoters," *RNA*, vol. 16, no. 3, pp. 495–505, 2010.

[8] Simon J.D. Prince, *Understanding Deep Learning*, The MIT Press, 2023.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017.

[10] Andreas Veit, Michael Wilber, and Serge Belongie, "Residual networks behave like ensembles of relatively shallow networks," *arXiv*, 2016.

[11] Dongfang Wang and Jin Gu, "Vasc: Dimension reduction and visualization of single-cell rna-seq data by deep variational autoencoder," *Genomics, Proteomics Bioinformatics*, vol. 16, no. 5, pp. 320–331, October 2018.

[12] Yuma Ichikawa and Koji Hukushima, "Learning dynamics in linear vae: Posterior collapse threshold, superfluous latent space pitfalls, and speedup with kl annealing," *arXiv*, 2023.

[13] Jing Jiang, Junlin Xu, Yuansheng Liu, Bosheng Song, Xiulan Guo, Xiangxiang Zeng, and Quan Zou, "Dimensionality reduction and visualization of single-cell rna-seq data with an improved deep variational autoencoder," *Briefings in Bioinformatics*, vol. 24, no. 3, May 2023.

DECLARATION OF USE OF GENERATIVE AI

This declaration **must** be filled out and included as the **final page** of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- ChatGPT 5
- Google Gemini

Describe how the tools were used:

What did you use the tool(s) for? Mostly coding and fixing errors in the code, as well as setting up environment.

At what stage(s) of the process did you use the tool(s)? Development of code (Feedforward architecture search, and transformer-related scripts) and debugging.

How did you use or incorporate the generated output? We used some scripts generated by AI, always checking them and adapting them if necessary. However, not all code is AI-generated, as we also took inspiration from the exercises of the course.

A. ADDITIONAL FIGURES

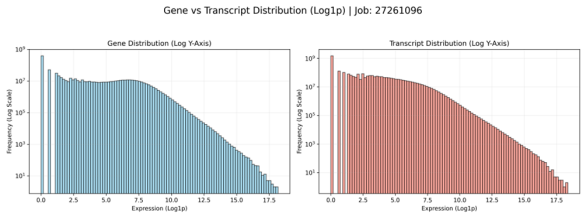


Fig. 5: Histogram showing distribution of log1p-transformed counts of genes (blue) and transcripts (red) in the bulk datasets. X-axis spans the log1p-transformed count values of all the genes or transcripts, and the y-axis spans the frequency of said genes or transcripts at the corresponding transformed count value. The y-axis is log-scaled.

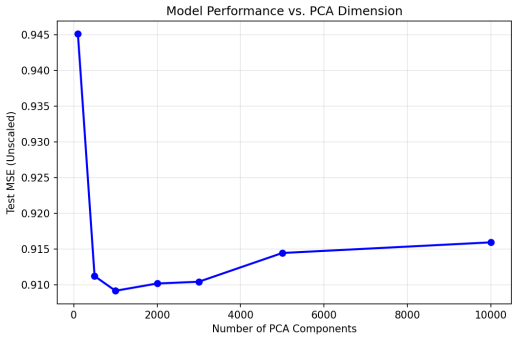


Fig. 6: MSE value of PCA-based FFNN models when trained using different numbers of PCA components. Hyperparameters were fixed in all models.